Task 1

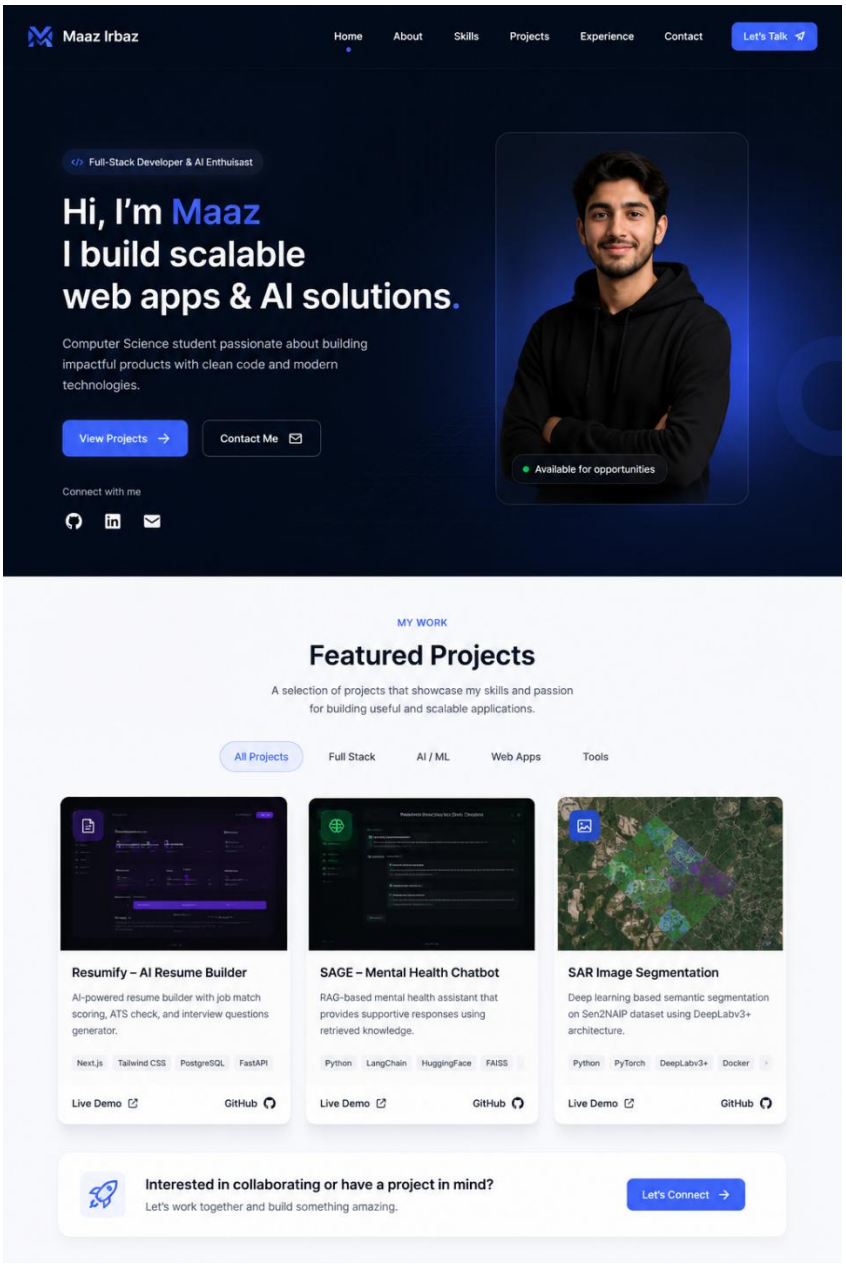
GPT prompt:
Act as a senior full-stack architect. I'm building a personal portfolio website with a Contact Us system for a MERN-stack internship task. Stack: Next.js App Router, Supabase (DB + Auth), Prisma, Resend for email, Google reCAPTCHA v3, deployed on Vercel. Give me: (1) a page/section list for the portfolio (Hero, About, Skills, Projects, Experience/Education, Contact, Footer), (2) the database flow from form submit → Prisma → Supabase → Resend email, (3) the authentication flow for a single seeded Admin user logging into a protected dashboard, (4) a deployment plan (repo → env vars → Vercel → production branch). Format as a short structured plan I can paste into a doc.

Result:
https://chatgpt.com/share/6a57996b-d138-83ee-9848-786743e2cd53

GPT prompt:

Based on the plan above, describe a clean, modern portfolio design concept — color palette, typography style, layout for Hero and Projects sections. Then generate a single reference image showing the overall portfolio layout (Hero + Projects section) in this style.

Result:

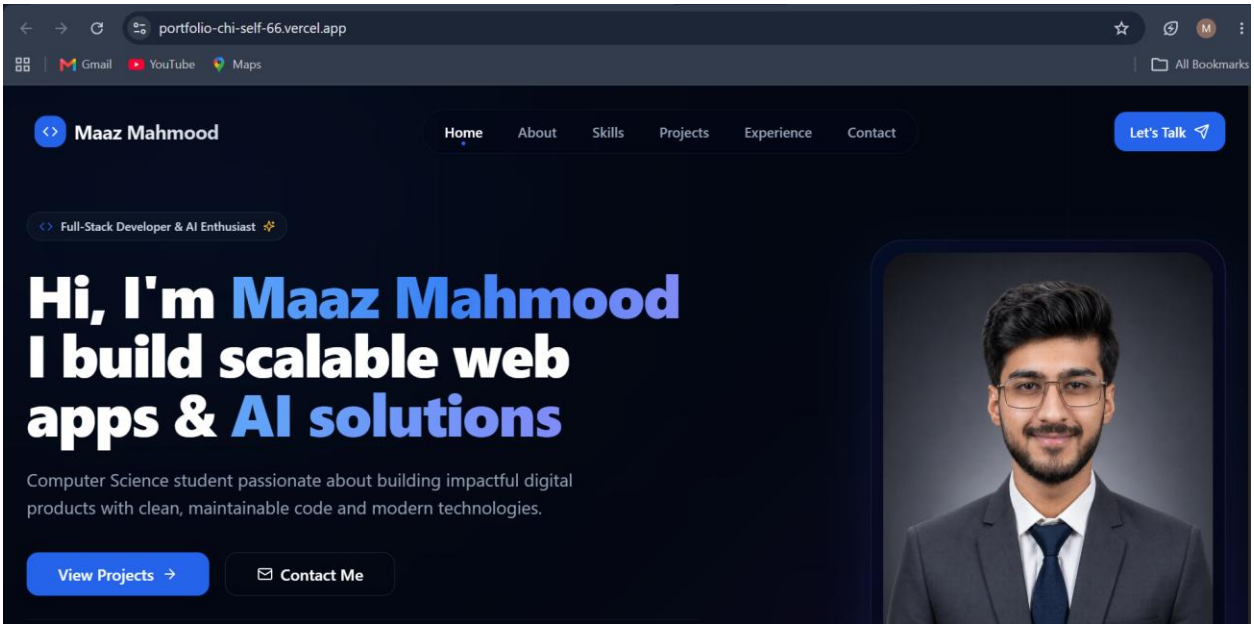

Maaz Irbaz
Home
About
Skills
Projects
Experience
Contact
Let's Talk
Full-Stack Developer & AI Enthusiast
Hi, I'm Maaz
I build scalable
web apps & AI solutions.
Computer Science student passionate about building impactful products with clean code and modern technologies.
View Projects
Contact Me
Connect with me
Available for opportunities
MY WORK
Featured Projects
A selection of projects that showcase my skills and passion for building useful and scalable applications.
All Projects
Full Stack
AI / ML
Web Apps
Tools
Resumify – AI Resume Builder
AI-powered resume builder with job match scoring, ATS check, and interview questions generator.
Next.js
Tailwind CSS
PostgreSQL
FastAPI
Live Demo
GitHub
SAGE – Mental Health Chatbot
RAG-based mental health assistant that provides supportive responses using retrieved knowledge.
Python
LangChain
HuggingFace
FAISS
Live Demo
GitHub
SAR Image Segmentation
Deep learning based semantic segmentation on Sen2NAIP dataset using DeepLabv3+ architecture.
Python
PyTorch
DeepLabv3+
Docker
Live Demo
GitHub
Interested in collaborating or have a project in mind?
Let's work together and build something amazing.
Let's Connect

Task 2

Cursor Prompt:
Scaffold a Next.js (App Router, TypeScript, Tailwind, shadcn/ui) project for a personal portfolio. Create sections/components: Hero, About, Skills, Projects, Experience, Contact, Footer. Make it fully responsive and accessible. Use placeholder content for now.
Cursor Prompt:
Add a Contact Us section with a form containing name, email, phone/subject, and message fields. Add client-side validation (required fields, valid email format, min message length) with inline, user-friendly error messages. Don't wire it to a backend yet just local state and a console.log on submit.

Task 3

deployed



Task 4

GPT prompt:
Design a Prisma schema for a Contact model with fields: id, name, email, phone (nullable), subject (nullable), message, status (enum: Pending, Done, Completed, Resolved — default Pending), created_at, updated_at. Also design a Profile model linked to Supabase Auth (auth_user_id, name, email, role). Output valid Prisma schema syntax.

Cursor prompt:
Set up Prisma in this Next.js project connected to Supabase Postgres. Add this schema: schema in docs. Generate the migration, run prisma generate, and create a server action submitContact that validates and inserts a row into Contact with status "Pending". Wire the Contact form's onSubmit to this server action, with loading/success/error UI states.

Task 5

Cursor prompt:
Integrate the Resend SDK. After a Contact row is successfully inserted, send an email to my admin email using Resend with subject "New Contact Query" and body including the sender's name, email, and a short summary of the message. Wrap the Resend call in try/catch so a failed email never blocks the form submission. show the same success UI to the user either way, but log the error server-side.

Task 6:

GPT prompt:
Explain the correct approach to seed exactly one Admin user in Supabase: creating the user in Supabase Auth via the admin API, then inserting a matching row in the Profile table with role "Admin", using a Node/TS script. Outline the script's steps before I generate the code.

Cursor prompt:
Read the structure and Create scripts/seed-admin.ts that uses the Supabase service role key to create one Admin user in Supabase Auth (email + password from env vars) and inserts a linked row into the Profile table with role "Admin". Add an npm script seed:admin to run it with ts-node.

Cursor prompt:
Build a login page using Supabase Auth (email/password) with clean validation and error messages. After login, create a protected /dashboard layout with a sidebar (Dashboard, Contact Queries) that only Admin can access — redirect unauthenticated users to /login. Build the Dashboard page showing stats: total contacts, pending count, resolved/completed count, and the 5 most recent contacts.

Cursor prompt:
Create a Contact Queries page listing all Contact rows in a table, with a status dropdown per row (Pending, Done, Completed, Resolved) that updates the row via a server action. Add a working Logout button in the sidebar.

Task 7:

Cursor prompt:
Add Google reCAPTCHA v3 to the login page: load the script client-side, get a token on submit, and verify it server-side against the Google siteverify API before processing login, reject with a friendly message if the score is too low. Then add IP-based login rate limiting using a LoginAttempt table (ip_address, attempts, last_attempt_at, blocked_until): if the same IP fails more than 5 times, block further attempts for a cooldown period and show a clear "too many attempts, try again in X minutes" message. Make sure no internals (attempt counts, IPs) leak to the frontend.

Task 8

Cursor prompt:

Review the project for cleanup: remove console.logs, unused code, and confirm all env vars are referenced correctly. Then give me the git commands to create a production branch, merge main into it, and push. So I can connect it as the Vercel production branch.